When LLM Labels Fail Together

HanRui Zhang
High School Student, Marc Garneau Collegiate Institute
Toronto, Canada
daniel21cn2016@gmail.com

***Abstract*—Large language models can label many texts in one prompt, saving repeated instructions and inference cost. Because those items share one generated response, their errors may be dependent. We measure this dependence with one-target controls that show the same neighboring texts in the same order. Across a batch-16 grid of four open-weight model families and four text-classification tasks, mean excess error covariance (the added tendency for errors to occur together) was 0.00961 (one-sided randomization $p = 0.0002$). All 16 estimates were positive, and 13 exceeded the prespecified practical threshold of 0.002. In four selected model–task combinations, pairwise dependence weakened as batch size increased, but total dependence raised the estimated variance from 1.08 to 2.38 times the independent-label value. In a secondary analysis completed after the main experiment, ignoring call membership made the results look too certain in three of four combinations; intervals based on whole calls came much closer to their intended 95% coverage. Each batched annotation call should therefore be treated as a statistical cluster. Studies using batched labels should record which call and protocol produced each label and account for within-call dependence when estimating uncertainty.**

***Index Terms*—large language models, data annotation, batch prompting, dependent errors, uncertainty estimation**

I. Introduction

Large language models (LLMs) can label text quickly and at low cost. Prior studies compare their labels with those of crowd workers and experts [1], [2]. Larger evaluations show that quality varies across tasks and datasets and still requires human validation [3], [4]; even meaning-preserving prompt changes can alter performance [5]. Batch-prompting studies focus on efficiency and per-item accuracy [6], [7]. Related work reports interference between items, correlated errors across models, and unreliable uncertainty estimates under other forms of dependence [8], [9], [10], [11]. This leaves a missing question: do items labeled in the same response also tend to be wrong together, making confidence intervals too narrow?

Overall accuracy does not reveal whether errors are independent. When 16 texts share one prompt and response, the model can carry an interpretation or formatting decision across the call. Positively correlated errors leave a dataset with less independent information than its row count suggests. Confidence intervals that treat the labels as independent may then be too narrow even when their point estimates are unbiased. Classical methods for clustered data address this gap [12], [13].

We test whether requesting multiple labels in one call makes errors co-occur across items (Fig. 1). Each joint prompt is paired with an exact-context control showing the same documents in the same order but requesting only one target label. This keeps context fixed while changing whether labels share a response. The study includes a 4×4 model–task grid at batch size 16, five batch sizes from 2 to 32 in one prespecified combination per model family, and a recovered analysis comparing item-level confidence intervals with intervals based on calls.

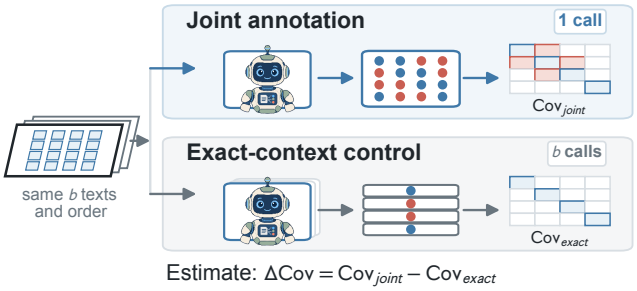


Fig. 1. Matched-context design for a batch of b texts. The joint condition returns b labels in one call, whereas the control generates each target label in one of b separate calls using the same visible context. Covariance matrices are schematic; the reported measure subtracts exact-context covariance from joint-call covariance. The 4×4 model–task grid and recovered coverage analysis use $b = 16$; the batch-size study uses $b \in \{2, 4, 8, 16, 32\}$ in four prespecified model–task combinations.

The study makes two contributions. First, it measures excess error covariance across a balanced 4×4 grid and separates weaker pairwise dependence from greater total information loss as batch size grows. Second, a recovered repeated-sampling analysis measures how often 95% confidence intervals miss their target when call membership is ignored. We recommend storing a call identifier with every LLM-generated label and using it to estimate uncertainty.

II. Related Work

LLMs are increasingly evaluated as substitutes or complements for human annotators. Gilardi et al. found that ChatGPT could outperform crowd workers on several text-annotation tasks, and Tan et al. organize the broader literature around annotation generation, assessment, and downstream use [1], [2]. Larger evaluations add important qualifications. Ziems et al. found substantial variation across models and computational social science tasks. Pangakis and Wolken found that performance varied across 27 tasks from 11 protected datasets and argued for validation against human labels [3], [4]. Even minor prompt-format changes can alter model performance [5]. These studies show that LLM annotation quality depends

on the task and protocol, but they mainly measure item-level accuracy, agreement, or sensitivity. Kim et al. measure error correlation across distinct models [9]; we measure dependence among items labeled within one shared generation.

Batch prompting is the closest setting to our study. Cheng et al. evaluate its token, time, and accuracy tradeoffs across ten datasets [6]. Pipal et al. study annotation more directly across eight production models, four tasks, and batch sizes up to 1,000 [7]. Both focus on efficiency and per-item accuracy rather than whether labels sharing a response also share errors. Lin et al. show that batch-prompt performance depends on item position and order because earlier outputs change the decoder context [14]. Yue and Yao show that a malicious instruction in one item can disrupt other outputs in the same batch [8]. We study ordinary classification inputs and compare them with target-only calls that show the same context.

Methods for correlated data are also established outside batched annotation. Gligorić et al. use LLM confidence and selective human annotation to construct confidence intervals with correct coverage [15]. Llewellyn et al. cluster semantically dependent benchmark prompts, while Zeng et al. address missing responses when observations from the same user are dependent [10], [11]. These studies address annotation error, prompt structure, or user-level dependence. We test dependence created when multiple labels share one generation. Standard methods already handle known clusters [12], [13], [16], [17]. Our contribution is a matched estimate of call-associated error covariance and a repeated-sampling evaluation of confidence-interval coverage when call membership is discarded.

III. Experimental Design

A. Models and Tasks

We evaluated Qwen3-8B, Granite-3.3-8B-Instruct, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3, representing four similarly sized open-weight model families [18], [19], [20], [21]. The tasks were toxicity classification on Civil Comments [22], natural-language inference on MultiNLI [23], sentiment classification on IMDb [24], and topic classification on AG News [25]. We call each model–task combination a *cell*. Civil Comments scores ≥ 0.5 were labeled toxic. After fixed exclusions, we hash-ordered candidates and divided the confirmation set into 16 equal-sized text-length strata. Dataset labels served as references, so an "error" means disagreement with that label, not necessarily an objectively wrong prediction.

Every model used the same instruction template, JSON grammar, parser, greedy decoding, and 32,768-token context limit. Each task had 8,192 fixed items, a count chosen before observing joint outputs using solo-prompt error rates and CPU-based power simulations. No cell was removed based on accuracy or parse rate.

B. Joint and Exact-Context Conditions

For each cell, we formed length-balanced groups of 16 items under three independent randomized assignments. In the *joint* condition, one prompt requested labels for all 16 items. In the *exact-context* condition, the prompt showed those same 16 items in the same order but requested only one target label. Repeating the prompt for all targets produced one label per item in separate calls. This control keeps the visible context constant while changing whether labels share a generated response.

Let $e_i \in \{0, 1\}$ indicate disagreement with the reference label, and let $\bar{e}$ be the mean error for each model, task, condition, and assignment. We placed these fixed errors into matched 16-item groups representing either one joint call or 16 corresponding exact-context calls. Within each group, we averaged $(e_i - \bar{e})(e_j - \bar{e})$ over distinct item pairs, then averaged the three assignment contrasts. The primary statistic for each cell was

$$\Delta = \mathrm{Cov}_{\mathrm{joint}}(e_i, e_j) - \mathrm{Cov}_{\mathrm{exact}}(e_i, e_j). \tag{1}$$

A positive Δ means that errors co-occur more often when labels share a response than when they only share context. We tested a *sharp null*: with outputs fixed, group assignment has no effect. In 4,999 paired replays, we permuted items within assignment, length stratum, and execution block while preserving group size and repeated appearances; both conditions used the same regrouping. We calculated upper-tail p-values and applied Holm's adjustment to control for testing 16 cells. The 95% randomization-based interval subtracts mean-centered replay quantiles from the estimate and applies only to the fixed outputs and replay design. Before observing joint results, we set $\Delta \geq 0.002$ as the practical threshold and evaluated it separately from statistical rejection. The combined estimate gives every cell equal weight and applies only to the tested models, tasks, and items.

We retained malformed outputs under four prespecified rules: count unresolved targets as correct, count them as errors, drop only unresolved targets, or drop their entire calls. Testing all four rules prevents selective parsing choices from creating the result.

C. Batch-Size Robustness

We repeated the joint-versus-exact comparison in one prespecified cell per model family at batch sizes 2, 4, 8, 16, and 32. Each size used all 8,192 items, three nested randomized assignments, and the same output format. Dividing covariance by its condition-specific Bernoulli variance produced excess within-call error correlation. We regressed this value on $\log_2$ batch size for each cell and averaged the four slopes. Because items recur across assignments, these slopes are descriptive rather than independent replicates. A size-32 block-bootstrap sensitivity separately resamples assignments and breaks recurring item identity, so it does not support the main uncertainty claims. We converted pooled excess correlation ρ_Δ into a design effect, the factor by which dependence increases variance: $1 + (b-1)\rho_\Delta$. The effective sample size, $8{,}192/[1 + (b-1)\rho_\Delta]$, is the equivalent number of independent labels.

D. Recovered Repeated-Sampling Sensitivity

The repeated-sampling analysis tested whether call dependence changes uncertainty in the mean disagreement

rate. Granite/IMDb, Qwen/Civil Comments, Llama/Civil Comments, and Mistral/MultiNLI were selected by a fixed task order and solo-prompt results before we inspected covariance. Each of 250 reference and 400 evaluation replicates sampled 512 item appearances and generated 32 new 16-item calls. The reference replicates estimated the target disagreement rate. Each evaluation replicate produced three nominal 95% intervals: a Wald interval treating 512 errors as independent, a t interval over 32 call means, and a whole-call bootstrap interval. Coverage is the percentage containing the target rate; a calibrated 95% method should cover it about 95% of the time. With 400 replicates, its maximum Monte Carlo standard error is 2.5 percentage points. Prespecified precision and stability checks supported assessing undercoverage for Granite, Qwen, and Mistral.

We recovered 1,095 of 83,200 calls (1.3%) with malformed or truncated responses. Reruns kept the original prompt, model, seed, numerical precision, runtime, and decoding settings; only the output format and maximum length changed. We retained all successful original outputs. Because the original coverage analysis could not be completed, the recovered analysis is a secondary, post-execution sensitivity. The primary covariance analysis was unchanged and includes all prespecified malformed-output rules.

E. Artifact Availability

The reviewer artifact contains analysis code, frozen protocols and prompts, dataset indices, and analysis-ready error arrays. It reproduces the reported analyses. Because it excludes dataset text, raw model responses, and cloud execution records, it cannot verify model execution or parsing. It is available at https://anonymous.4open.science/r/batched-annotation-D206/README.md.

IV. Results

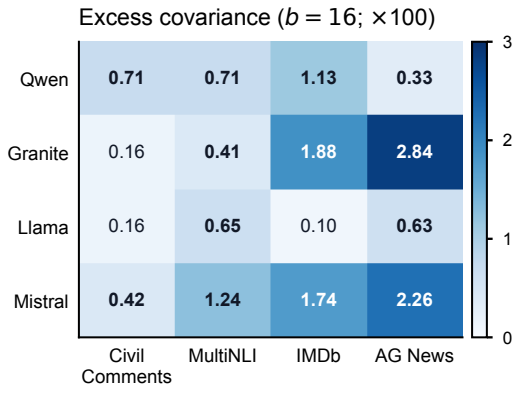


Fig. 2. Joint-minus-exact error covariance at batch size 16 (×100; 8,192 items per cell). Bold estimates meet the prespecified 0.002 practical threshold; all 16 cells reject the sharp null after Holm adjustment.

A. Dependence Recurs Across Models and Tasks

The mean excess covariance across cells was 0.00961, with a 95% randomization-based interval of [0.00936, 0.00986] and one-sided replay $p = 0.0002$. All 16 cells rejected the sharp null after Holm adjustment, and 13 also met the separate 0.002 practical threshold. Those 13 cells covered all four model families and all four tasks (Fig. 2). Every mean calculated after removing one model or one task remained positive, so no single family or dataset drove the combined result.

All four prespecified rules for malformed outputs gave combined excess covariance from 0.00752 to 0.01550, always above the practical threshold. A sensitivity using 3,002 recovered calls gave 0.00790. Thirteen cells passed a combined rule requiring both a large enough effect and rejection by the replay test, although two differed from the primary set. All 48 assignment-specific contrasts were positive, and means calculated after removing one assignment ranged from 0.00935 to 0.00992. Across cells and assignments, standardized covariance averaged 0.0557 in the joint condition and 0.0014 in the exact-context condition. These values gave mean design effects of 1.835 and 1.021 and effective sample sizes of 5,022 and 8,105 per 8,192 labels, respectively.

A post-hoc Civil Comments sensitivity kept four-cell covariance positive at thresholds 0.4 (0.00343), 0.5 (0.00464), and 0.6 (0.00626), and after excluding 509 ambiguous items (0.00524). Both Civil Comments batch-size slopes remained negative, and the repeated-sampling conclusions were qualitatively unchanged.

The information loss varied across settings. A descriptive calculation assuming equal correlation among all pairs within a call gave cell-level design effects from 1.14 to 2.87. For 8,192 labeled rows, these values correspond to effective sample sizes from 7,162 down to 2,850. As a negative control, fake batches assembled from separately generated solo labels had covariance between −0.00086 and 0.00082 across the 16 cells. Regrouping independent labels therefore did not reproduce the dependence observed when labels shared a response.

B. Dependence Persists but Does Not Grow Uniformly with Batch Size

Standardized excess dependence averaged across the four cells was positive at every tested size: 0.079 at batch size 2, 0.081 at size 4, and 0.045 at size 32 (Fig. 3). The mean slope was −0.00910 per doubling of batch size. It was negative in each complete assignment (−0.00960, −0.00879, and −0.00889). Dependence between pairs therefore decreased on average after size 4, although the three-assignment comparison is descriptive.

Three cells had negative slopes, whereas Granite/IMDb had a positive slope; no single trend described every selected cell.

Weaker pairwise dependence did not prevent the total variance penalty from growing. The descriptive design effect increased from 1.079 at batch size 2 (assignment range 1.076–1.083) to 2.381 at size 32 (2.328–2.473). Relative to the exact-context baseline, the corresponding effective sample size fell from 7,589 to 3,440 out of 8,192 rows. Each pair became less dependent on average, but every label shared its call with more labels.

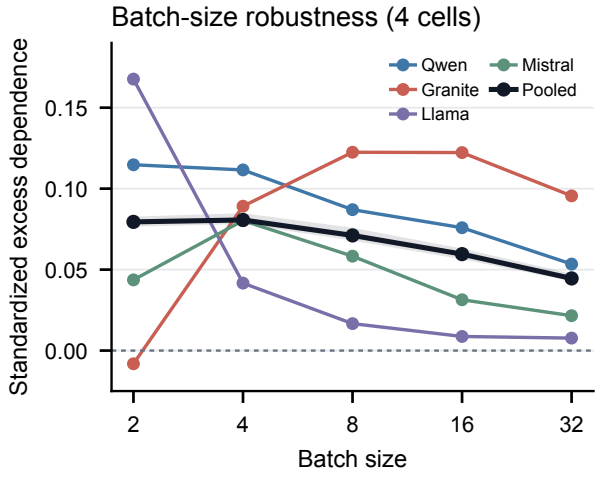


Fig. 3. Standardized joint-minus-exact dependence across batch sizes 2–32 in four prespecified cells. The black line is the equal-cell mean; shading spans the three complete randomized assignments.

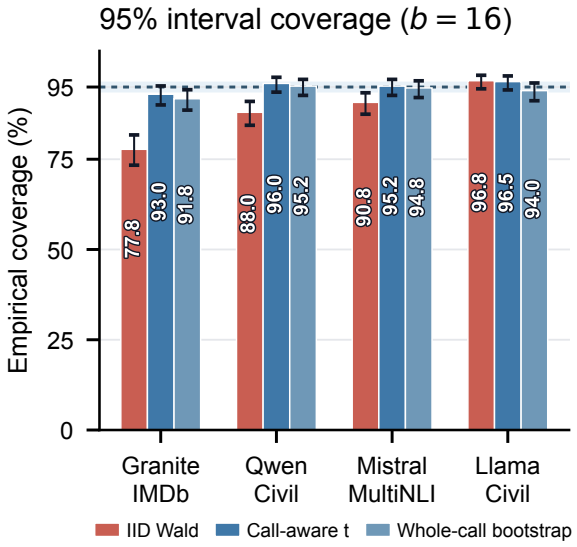


Fig. 4. Recovered post-execution analysis of coverage: the share of 400 repeated confidence intervals containing the target error rate at batch size 16. Error bars are exact 95% binomial intervals; shading marks 94–96% coverage.

C. Recovered Sensitivity: Ignoring Calls Can Understate Uncertainty

We next tested whether the observed covariance made 95% confidence intervals miss their target more often than expected. Figure 4 reports coverage with exact binomial intervals. For Granite/IMDb, the item-level method covered the target in only 77.8% of replicates, and the observed variance was 2.71 times the value assumed by that method. The call-aware t interval reached 93.0% coverage. Qwen/Civil Comments showed the same pattern, improving from 88.0% to 96.0%.

Mistral/MultiNLI had milder undercoverage (90.8%) and improved to 95.3% with the call-aware interval. Llama/Civil Comments showed no item-level undercoverage. Whole-call bootstrap coverage was 91.8%, 95.3%, 94.8%, and 94.0% for Granite, Qwen, Mistral, and Llama, respectively. Wilson-score intervals reproduced the item-level result, so the undercoverage did not result from using the Wald formula near a probability boundary. Undercoverage varied across cells, but the risk created by positive dependence cannot be diagnosed from the number of labeled rows alone.

V. Discussion

The unit that generates an annotation can also create statistical dependence. A joint prompt may establish a temporary decision rule, interpretation, or output pattern that affects several labels at once. Exact-context controls show that longer prompts or neighboring documents alone do not explain the observed covariance. We therefore analyze each joint call as a cluster while leaving the causal mechanism unresolved. An exploratory analysis found no clear linear trend in the difference in error rates by output position, while errors at nearby positions had stronger covariance. This pattern does not identify a mechanism.

The batch-size result separates dependence between pairs from the total information loss within a call. Dependence remained detectable even in small batches. It decreased in three selected cells and increased in one across the tested range. The design effect still rose with batch size because the number of other labels sharing each call grew faster than pairwise dependence declined. Reducing batch size can reduce this penalty, but the remaining labels may still be dependent.

This distinction matters when estimating means or proportions from batched labels. Positive covariance within a call can make formulas that assume independent rows underestimate variance. Researchers should retain original and parent call IDs, attempt and recovery status, model and protocol versions, decoding settings, batch size, and output position. These records permit call-aware analysis and auditing. Call-level t intervals, cluster-robust standard errors, or whole-call bootstrap resampling can account for call clustering [13], [16], [17]. The choice depends on the number and structure of independent calls, and studies with few calls require small-sample methods. Sources, users, conversations, or documents may add other forms of clustering that the analysis must also address. These methods change estimated uncertainty, not label accuracy. Effects on other downstream estimates remain untested.

The study has four limitations. First, it evaluates four open-weight models near the 7–8B scale and four English classification tasks; proprietary, larger, multilingual, generative, and subgroup-specific behavior may differ. Second, the batch-size experiment uses four different cells and sizes 2–32, so it cannot separate model-family effects from task effects or establish a universal scaling law. Third, the matched control identifies

the combined effect of joint annotation but cannot determine whether multi-target instructions, shared-response decoding, or another component produces the dependence. Fourth, the repeated-sampling study required post-execution recovery of 1,095 calls with a modified output format and response-length limit. The unrepaired subsets gave similar item-level coverage: 78.9% for Granite/IMDb (105/133 replicates) and 88.4% for Qwen/Civil Comments, compared with recovered values of 77.8% and 88.0%. These unrepaired subsets were too small for the prespecified decision rule, so the coverage result remains secondary sensitivity evidence.

VI. Conclusion

The primary grid showed that errors occurred together more often when labels shared a call across four model families and four tasks at batch size 16. In the descriptive four-cell batch-size analysis, dependence between pairs decreased on average, but the total variance penalty increased. In the recovered sensitivity, treating 512 labels as independent produced confidence intervals that missed their target too often in three cells. Intervals based on the 32 call means were much closer to their intended 95% coverage. Studies using batched labels should record the call and protocol that produced each label, report dependence separately from accuracy, and account for every source of grouping in the analysis.

References

[1] F. Gilardi, M. Alizadeh, and M. Kubli, "ChatGPT outperforms crowd workers for text-annotation tasks," *Proceedings of the National Academy of Sciences*, vol. 120, no. 30, p. e2305016120, 2023.

[2] Z. Tan, D. Li, S. Wang, A. Beigi, B. Jiang, A. Bhattacharjee, M. Karami, J. Li, L. Cheng, and H. Liu, "Large language models for data annotation and synthesis: A survey," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 930–957.

[3] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, "Can large language models transform computational social science?" *Computational Linguistics*, vol. 50, no. 1, pp. 237–291, 2024.

[4] N. Pangakis and S. Wolken, "Keeping humans in the loop: Human-centered automated annotation with generative AI," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 19, no. 1, 2025, pp. 1471–1492.

[5] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting," in *International Conference on Learning Representations*, 2024.

[6] Z. Cheng, J. Kasai, and T. Yu, "Batch prompting: Efficient inference with large language model APIs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2023, pp. 792–810.

[7] C. Pipal, E.-M. Vogel, M. Wack, and F. Esser, "Researchers waste 80% of LLM annotation costs by classifying one text at a time," *arXiv preprint arXiv:2604.03684*, 2026.

[8] M. Yue and Z. Yao, "Efficient but vulnerable: Benchmarking and defending LLM batch prompting attack," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4746–4761.

[9] E. M. Kim, A. Garg, K. Peng, and N. Garg, "Correlated errors in large language models," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 30 038–30 066.

[10] M. Llewellyn, I. Thornton, J. Bishop, and A. Gray, "Correcting prompt dependence in LLM benchmarks: A Bayesian hierarchical model with embedding-space clustering," *arXiv preprint arXiv:2510.05709*, 2025.

[11] Z. Zeng, D. Arbour, A. Feller, I. Dasgupta, A. R. Sinha, and E. H. Kennedy, "Handling missing responses under cluster dependence with applications to language model evaluation," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 56 527–56 556.

[12] K.-Y. Liang and S. L. Zeger, "Longitudinal data analysis using generalized linear models," *Biometrika*, vol. 73, no. 1, pp. 13–22, 1986.

[13] A. C. Cameron and D. L. Miller, "A practitioner's guide to cluster-robust inference," *Journal of Human Resources*, vol. 50, no. 2, pp. 317–372, 2015.

[14] J. Lin, M. Diesendruck, L. Du, and R. Abraham, "BatchPrompt: Accomplish more with less," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=Agyicd577r

[15] K. Gligorić, T. Zrnic, C. Lee, E. Candès, and D. Jurafsky, "Can unconfident LLM annotations be used for confident conclusions?" in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2025, pp. 3514–3533.

[16] J. G. MacKinnon, M. Ø. Nielsen, and M. D. Webb, "Cluster-robust inference: A guide to empirical practice," *Journal of Econometrics*, vol. 232, no. 2, pp. 272–299, 2023.

[17] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007.

[18] A. Yang, A. Li, B. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[19] IBM Granite Team, "Granite-3.3-8B-Instruct model card," Hugging Face, 2025. [Online]. Available: https://huggingface.co/ibm-granite/granite-3.3-8b-instruct

[20] A. Dubey, A. Jauhri, A. Pandey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[21] Mistral AI, "Mistral-7B-Instruct-v0.3 model card," Hugging Face, 2024. [Online]. Available: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3

[22] D. Borkan, L. Dixon, J. Sorensen, N. Thain, and L. Vasserman, "Nuanced metrics for measuring unintended bias with real data for text classification," in *Companion Proceedings of the 2019 World Wide Web Conference*, 2019, pp. 491–500.

[23] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2018, pp. 1112–1122.

[24] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150.

[25] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems 28*, 2015, pp. 649–657.